

Analyzing the Role of AI-Assisted Programming in Modern Software Development

Introduction

Artificial Intelligence (AI) has become an important part of modern software development. Today, many software developers use AI-powered tools to assist them in writing, testing, debugging, and improving computer programs. Popular tools such as ChatGPT, GitHub Copilot, Google Gemini, and Amazon Code Whisperer help developers complete programming tasks more efficiently by providing code suggestions, explanations, and error detection.

The use of AI-assisted programming has changed the way software is developed. It allows developers to save time, reduce repetitive work, improve code quality, and increase productivity. AI tools can also support beginners by explaining programming concepts and generating example code. However, AI-generated code is not always accurate or secure. Developers must review, test, and understand the generated code before using it in real software projects.

As AI continues to grow in the software industry, it is important for programmers to understand both its benefits and its limitations. While AI can improve development speed and efficiency, it should be used as a supporting tool rather than a replacement for programming knowledge and critical thinking.

This report explores the role of AI-assisted programming in modern software development. It explains the concept of AI-assisted programming, discusses the different types of AI coding tools, describes their advantages and limitations, and examines how AI can be used responsibly in software engineering. The report also includes a practical implementation using both traditional programming and AI-assisted programming, followed by a comparison of the two approaches and a discussion of the ethical considerations of using AI in software development.

Task 1: Conceptual Understanding

● What is AI-Assisted Programming?

AI-assisted programming is a way of using Artificial Intelligence (AI) tools to help programmers during software development. Instead of writing every line of code by themselves, developers can use AI tools to generate code, suggest solutions, find errors, and explain programming concepts. These tools help programmers complete their work more quickly and make coding easier, especially when working on large or complex projects. Some popular AI tools used for programming are ChatGPT, GitHub Copilot, Google Gemini, and Amazon Code Whisperer. Although these tools can save time and improve productivity, they are not always correct. Sometimes they generate incorrect or incomplete code, so programmers should always understand, review, and test the code before using it. Therefore, AI should be used as a helpful assistant, while the programmer remains responsible for the final software.

● Types of AI coding tools

Code generation tools help programmers write code automatically. The user gives instructions about what the program should do, and the AI creates the code. This helps save time and reduces the amount of coding that needs to be done manually. ChatGPT is an example of a code generation tool.

Code Completion Tools

Code completion tools suggest the next word, line, or part of the code while the programmer is typing. This helps developers write code faster and reduces typing mistakes. GitHub Copilot is a well-known code completion tool.

Debugging Tools

AI-Assisted Programming Assignment

Debugging tools help programmers find and fix errors in their programs. They explain what caused the error and suggest ways to solve it. This makes debugging easier and saves time.

Refactoring Tools

Refactoring tools help improve the structure of the code without changing how the program works. They make the code cleaner, easier to read, and easier to understand.

Documentation Tools

Documentation tools create comments and explanations for the code automatically. This helps programmers understand the code better and makes it easier for others to work on the same project in the future

● Advantages in software development

AI-assisted programming provides many benefits to software developers. First, it helps save time by generating code quickly and reducing the amount of manual work. It also increases productivity because developers can complete tasks faster. AI tools can help find and fix errors, making debugging easier and improving the quality of the code. They also help beginners learn programming by providing examples and explaining difficult concepts. In addition, AI reduces repetitive tasks, allowing developers to spend more time solving problems and creating better software. Overall, AI-assisted programming makes software development faster, easier, and more efficient.

● Risks and limitations

Although AI-assisted programming has many advantages, it also has some risks. Sometimes AI generates code that looks correct but contains mistakes or does not work properly. This is called hallucinated code. AI-generated code may also have security problems that can make software

unsafe if it is used without careful checking. In some cases, AI may not fully understand the user's instructions and produce code that does not match the required solution.

Another limitation is that programmers may become too dependent on AI. If they rely on AI for every task, they may not improve their programming and problem-solving skills. For this reason, AI should be used as a helpful tool rather than a replacement for learning. Developers should always review, understand, and test AI-generated code before using it in real software projects.

Task 2: Practical Implementation

1. Select a programming problem of moderate complexity

Library Management System

2. Develop the solution using TWO approaches:

Version A

```
import java.util.ArrayList;
import java.util.Scanner;
class Book {
    int id;
    String title;
    String author;
    Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }
}

public class LibraryManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> books = new ArrayList<>();
        int choice = 0;
        while (choice != 5) {
            System.out.println("\n Library Management System ");
            System.out.println("1. Add Book");
            System.out.println("2. View Books");
            System.out.println("3. Search Book");
```

```

System.out.println("4. Delete Book");
System.out.println("5. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
if (choice == 1) {
    System.out.print("Enter Book ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Enter Book Title: ");
    String title = sc.nextLine();
    System.out.print("Enter Author Name: ");
    String author = sc.nextLine();
    Book b = new Book(id, title, author);
    books.add(b);
    System.out.println("Book added successfully.");
} else if (choice == 2) {
    if (books.size() == 0) {
        System.out.println("No books available.");
    } else {
        for (int i = 0; i < books.size(); i++) {
            System.out.println("-----");
            System.out.println("Book ID : " + books.get(i).id);
            System.out.println("Title   : " + books.get(i).title);
            System.out.println("Author       :      " +
books.get(i).author);
        }
    }
} else if (choice == 3) {
    System.out.print("Enter Book ID to search: ");
    int searchId = sc.nextInt();
    boolean found = false;
    for (int i = 0; i < books.size(); i++) {
        if (books.get(i).id == searchId) {
            System.out.println("Book Found");
            System.out.println("Book ID : " + books.get(i).id);
            System.out.println("Title   : " + books.get(i).title);
            System.out.println("Author       :      " +
books.get(i).author);
            found = true;
        }
    }
    if (found == false) {
        System.out.println("Book not found.");
    }
} else if (choice == 4) {
    System.out.print("Enter Book ID to delete: ");
    int deleteId = sc.nextInt();
    boolean deleted = false;
    for (int i = 0; i < books.size(); i++) {
        if (books.get(i).id == deleteId) {

```

AI-Assisted Programming Assignment

```
        books.remove(i);
        deleted = true;
        break;
    }
}
if (deleted == true) {
    System.out.println("Book deleted successfully.");
} else {
    System.out.println("Book not found.");
}
} else if (choice == 5) {
    System.out.println("Program closed.");
} else {
    System.out.println("Invalid choice.");
}
}
sc.close();
}
```

Version B

```
import java.util.ArrayList;
import java.util.Scanner;

class Book {

    private int id;
    private String title;
    private String author;
    private boolean issued;

    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.issued = false;
    }
    public int getId() {
        return id;
    }
    public void setDetails(String title, String author) {
        this.title = title;
        this.author = author;
    }
    public void issueBook() {
```

```
        issued = true;
    }
    public void returnBook() {
        issued = false;
    }
    public void display() {
        System.out.println("-----");
        System.out.println("Book ID : " + id);
        System.out.println("Title   : " + title);
        System.out.println("Author  : " + author);

        if (issued) {
            System.out.println("Status   : Issued");
        } else {
            System.out.println("Status   : Available");
        }
    }
}
public class LibraryManagementSystemAI {

    static ArrayList<Book> books = new ArrayList<>();
    static Scanner input = new Scanner(System.in);
    public static void main(String[] args) {
        int choice;
        do {
            System.out.println("\n===== Library Management System =====");
            System.out.println("1. Add Book");
            System.out.println("2. View Books");
            System.out.println("3. Search Book");
            System.out.println("4. Update Book");
            System.out.println("5. Delete Book");
            System.out.println("6. Issue Book");
            System.out.println("7. Return Book");
            System.out.println("8. Exit");
            System.out.print("Enter your choice: ");
            choice = input.nextInt();
            switch(choice) {
                case 1:
                    addBook();
                    break;
                case 2:
                    viewBooks();
                    break;
                case 3:
                    searchBook();
                    break;
                case 4:
                    updateBook();
                    break;
                case 5:
```

```
        deleteBook();
        break;
    case 6:
        issueBook();
        break;
    case 7:
        returnBook();
        break;
    case 8:
        System.out.println("System closed.");
        break;
    default:
        System.out.println("Invalid choice.");
    }

    } while(choice != 8);
}

static void addBook() {
    System.out.print("Enter Book ID: ");
    int id = input.nextInt();
    input.nextLine();
    for(Book b : books) {
        if(b.getId() == id) {
            System.out.println("Book ID already exists.");
            return;
        }
    }
    System.out.print("Enter Book Title: ");
    String title = input.nextLine();
    System.out.print("Enter Author Name: ");
    String author = input.nextLine();
    books.add(new Book(id, title, author));
    System.out.println("Book added successfully.");
}

static void viewBooks() {
    if(books.isEmpty()) {
        System.out.println("No books available.");
    } else {
        for(Book b : books) {
            b.display();
        }
    }
}

static void searchBook() {
    System.out.print("Enter Book ID: ");
    int id = input.nextInt();
    for(Book b : books) {
        if(b.getId() == id) {
```



```
        b.display();
        return;
    }
}
System.out.println("Book not found.");
}
static void updateBook() {
    System.out.print("Enter Book ID: ");
    int id = input.nextInt();
    input.nextLine();
    for(Book b : books) {
        if(b.getId() == id) {
            System.out.print("Enter new title: ");
            String title = input.nextLine();
            System.out.print("Enter new author: ");
            String author = input.nextLine();

            b.setDetails(title, author);

            System.out.println("Book updated successfully.");
            return;
        }
    }

    System.out.println("Book not found.");
}
static void deleteBook() {
    System.out.print("Enter Book ID: ");
    int id = input.nextInt();
    for(int i = 0; i < books.size(); i++) {
        if(books.get(i).getId() == id) {
            books.remove(i);
            System.out.println("Book deleted successfully.");
            return;
        }
    }
    System.out.println("Book not found.");
}
static void issueBook() {
    System.out.print("Enter Book ID: ");
    int id = input.nextInt();
    for(Book b : books) {
        if(b.getId() == id) {
            b.issueBook();
            System.out.println("Book issued successfully.");
            return;
        }
    }
}
```

```

        System.out.println("Book not found.");
    }

    static void returnBook() {
        System.out.print("Enter Book ID: ");
        int id = input.nextInt();
        for(Book b : books) {
            if(b.getId() == id) {
                b.returnBook();
                System.out.println("Book returned successfully.");
                return;
            }
        }
        System.out.println("Book not found.");
    }
}

```

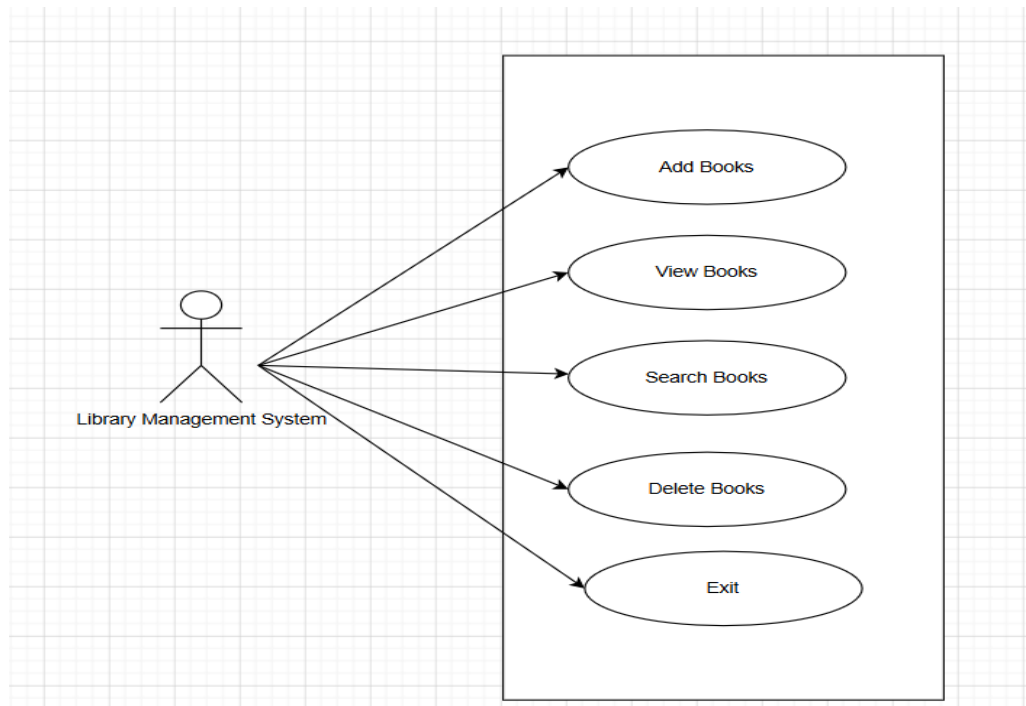
3. Compare both implementations based on

Comparison Criteria	VersionA (Manual Development)	VersionB (AI-Assisted Development)
Development Time	Took more time because the entire program was written manually. Designing the structure, writing code, and fixing mistakes required more effort.	Developed faster because AI assistance helped improve the code structure, add new features, and suggest better programming methods.
Code Quality	The code quality is acceptable for a simple system, but most of the code is written inside the main method. Adding new features can be difficult.	The code quality is better because the program is divided into separate methods and uses object-oriented programming concepts. It is cleaner, more organized, and easier to improve.

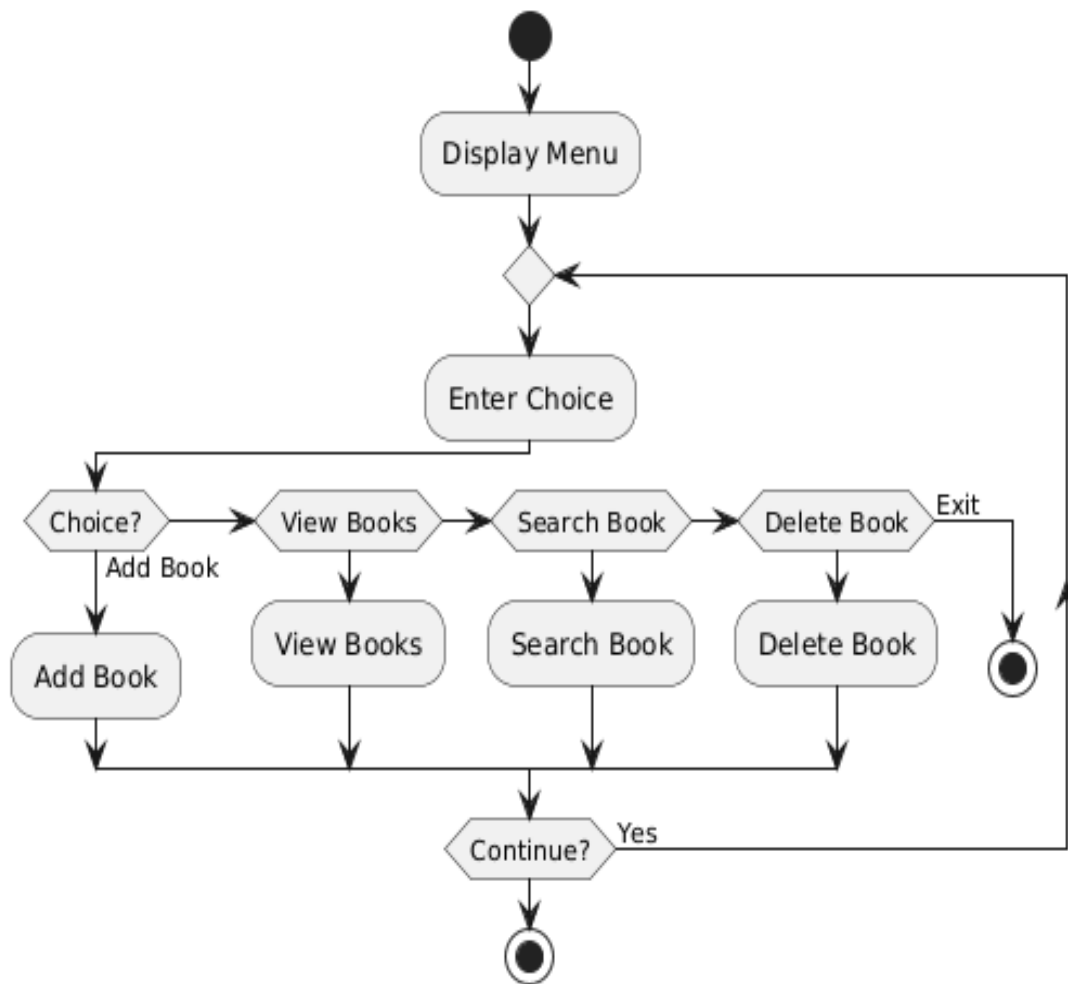
Comparison Criteria	VersionA (Manual Development)	VersionB (AI-Assisted Development)
Error Handling	Has limited error handling. It only checks basic situations, such as whether a book exists, and may not handle incorrect user input properly.	Provides better error handling by validating duplicate book IDs and improving control of user operations, reducing runtime errors.
Readability & Maintainability	Simple and easy for beginners to understand, but becomes difficult to maintain as more features are added because the code is not well organized.	More readable and maintainable because methods are separated, variable names are clearer, and the overall structure is well organized, making future modifications easier.

4. UML diagrams

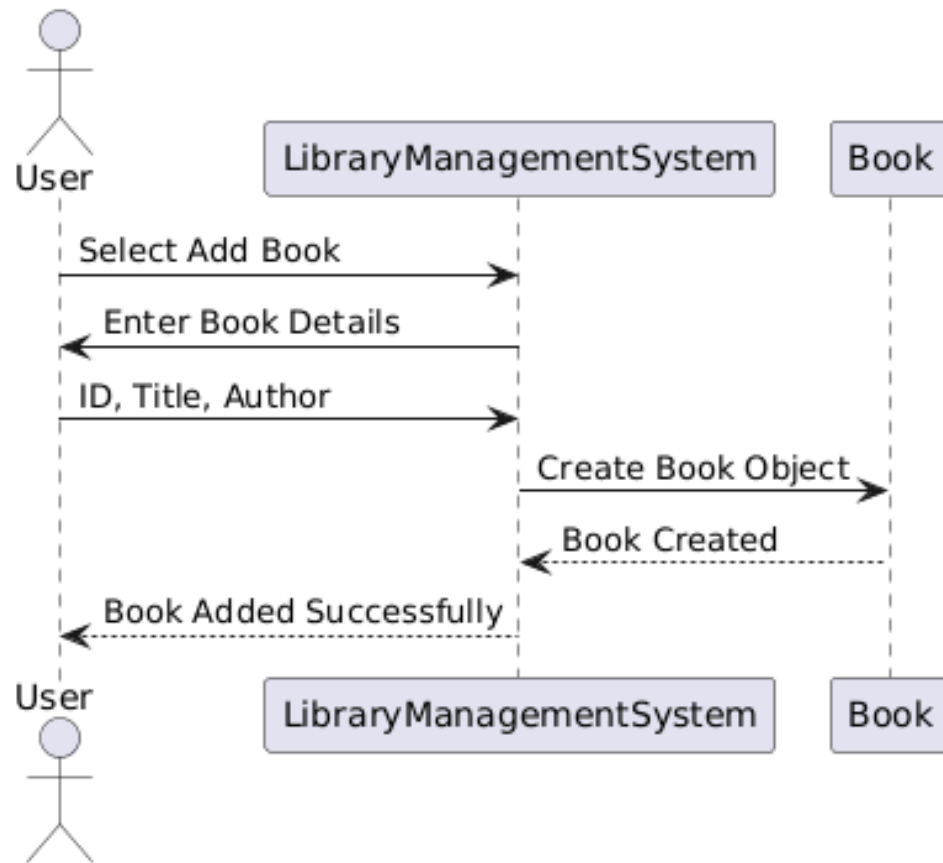
- Use Case Diagram



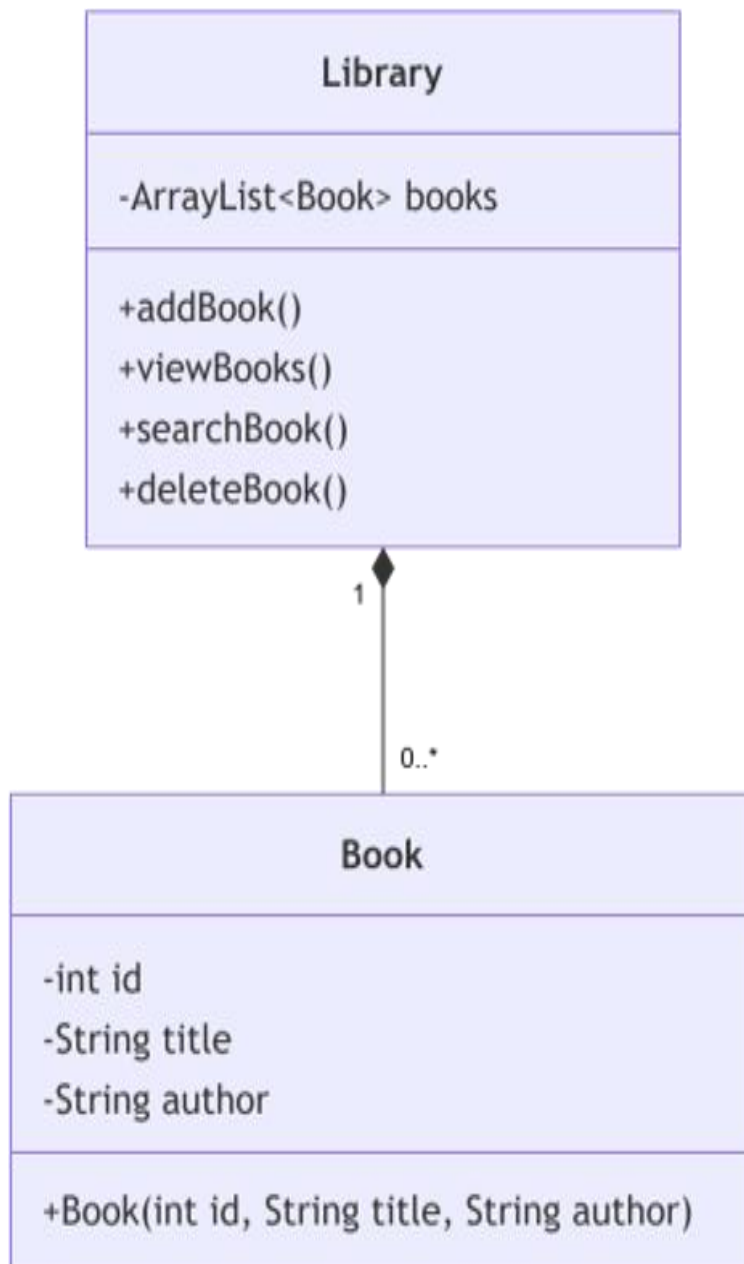
- Activity Diagram



- Sequence Diagram



- Class Diagram



Task 3: Critical Analysis

● How AI changed their coding approach

AI changed my coding approach by helping me write code more quickly and efficiently. It suggested ideas for improving the program structure and helped me solve coding problems faster. However, I still needed to understand the code and make changes where necessary.

● Whether AI improved or reduced their understanding

AI improved my understanding because it explained programming concepts and showed different ways to solve problems. By reading and testing the AI-generated code, I learned new Java techniques. However, if students depend on AI too much without studying the code, it may reduce their learning and problem-solving skills.

● In which cases AI should NOT be used

AI should not be used when students need to learn programming concepts on their own or during examinations where AI tools are not allowed. It should also not be used for developing software that requires highly secure, confidential, or safety-critical code without careful human review. AI-generated code should always be checked before it is used.

● Ethical considerations in AI-generated code

When using AI-generated code, it is important to use it responsibly. Students should not submit AI-generated work as their own without understanding it. Developers should also review AI-generated code for correctness, security, and originality to avoid errors, security risks, or copyright issues. AI should be used as a learning and support tool rather than a replacement for human knowledge.

Conclusion

AI is becoming an important tool in programming education and the software industry. It helps students and developers write code faster, find mistakes, and improve their work. However, it is important to understand the code instead of depending on AI all the time.

In my opinion, AI should be used as a helper, not as a replacement for programming skills. Students should use AI to learn and improve their knowledge, while developers should always check and test AI-generated code before using it. When used in the right way, AI can make programming easier, faster, and more effective.

GitHub repository

<https://github.com/thusharinilmini00041-jpg/LibraryManagementSystem/tree/main>

Name : W.K.T.Nilmini

Index : s17360

Reg. No. : 2023s20092